# Sprawozdanie z realizacji projektu nr 4:

# Steganografia obrazowa z wykorzystaniem najmniej znaczącego bitu (LSB)

Imię i nazwisko: Cyprian Magnucki 197697, Małgorzata Rogala 198123

Grupa: A2A

Przedmiot: Organizacja Systemów Komputerowych - Laboratorium

Data: 10.04.2026 r.

## 1. Sformułowanie zadania i założenia techniczne

Celem projektu było zaprojektowanie i zaimplementowanie narzędzia programistycznego umożliwiającego bezpieczne i dyskretne ukrywanie danych tekstowych wewnątrz plików graficznych. Wykorzystano technikę LSB (Least Significant Bit), która polega na zamianie najmniej znaczącego bitu składowej koloru piksela na bit pochodzący z wiadomości.

### Szczegółowe założenia projektowe:

1. **Formaty wejściowe:** Aplikacja akceptuje pliki .png, .jpg, .jpeg oraz .bmp.
2. **Format wyjściowy:** Ze względu na konieczność zachowania integralności bitowej, plik wynikowy jest zawsze zapisywany w formacie PNG (kompresja bezstratna).
3. **Nośnik danych:** Informacja jest zapisywana w kanale czerwonym (R) każdego piksela.
4. **Mechanizm zakończenia:** Strumień danych kończy się unikalnym markerem (delimiterem) ##### co pozwala na poprawną ekstrakcję tekstu bez konieczności znania jego długości przed procesem odczytu.
5. **Interfejs:** GUI zbudowane w oparciu o bibliotekę tkinter zapewniające podgląd statusu operacji.

## 2. Opis rozwiązań programowych i algorytmiki

### Reprezentacja danych i konwersja

Każdy znak tekstu jest konwertowany na 8-bitową liczbę binarną zgodnie ze standardem kodowania znaków. Jest to kluczowe, ponieważ algorytm operuje bezpośrednio na strumieniu bitów, a nie na całych bajtach.

### Algorytm modyfikacji bitów (Szyfrowanie)

Najważniejszym elementem kodu jest operacja bitowa wykonywana na składowej koloru. Zastosowano maskowanie bitowe:

- r & ~1: Operacja logicznego "AND" z maską 11111110. Powoduje to wyzerowanie ostatniego bitu składowej czerwonej, pozostawiając pozostałe 7 bitów bez zmian.
- | int(strumien_bitow[indeks]): Operacja logicznego "OR", która wstawia bit wiadomości (0 lub 1) w miejsce wcześniej wyzerowanego bitu.

### Kluczowy fragment implementacji modyfikacji pixela:

```
r, g, b = piksele[x, y]
```

Wyzerowanie ostatniego bitu i nadpisanie bitem wiadomości:

```
nowy_r = (r & ~1) | int(strumien_bitow[indeks])
piksele[x, y] = (nowy_r, g, b)
```

### Algorytm ekstrakcji (Odczyt)

Proces odczytu jest odwrotnością szyfrowania. Program iteruje po pikselach i wykonuje operację r & 1, która izoluje wyłącznie ostatni bit. Bity są zbierane w grupy 8-elementowe, a następnie rzutowane z powrotem na znaki ASCII. Proces kończy się w momencie wykrycia sekwencji #####.

## 3. Analiza pojemności nośnika

Jednym z kluczowych aspektów steganografii jest określenie pojemności informacyjnej nośnika. W przyjętym modelu wykorzystujemy 1 bit na każdy piksel obrazu.

### Obliczenia teoretyczne:

Każdy znak (wliczając spacje i znaki specjalne) zajmuje 8 bitów. Ponieważ jeden piksel przechowuje 1 bit informacji. Do ukrycia jednego znaku potrzebujemy 8 pikseli.

Wzór na maksymalną liczbę znaków (N):

$$N = \frac{Szerokość \times Wysokość}{8} - L_{znak\ kończący}$$

Gdzie długość znaku kończącego to w naszym przypadku 5 znaków.

### Przykłady pojemności dla popularnych rozdzielczości:

| **Rozdzielczość obrazu** | **Łączna liczba pikseli** | **Max. liczba znaków (ok.)** | **Przybliżona objętość tekstu** |
|---|---|---|---|
| **800 x 600** | 480 000 | 60 000 | ok. 15 stron maszynopisu |
| **1280 x 720 (HD)** | 921 600 | 115 200 | mała książka |
| **1920 x 1080 (FHD)** | 2 073 600 | 259 200 | obszerna powieść |

## 4. Dyskusja osiągniętych wyników

### Zalety aplikacji:

- **Wysoka dyskrecja:**
  Zmiana wartości koloru o 1 (np. z 255 na 254) skutkuje różnicą natężenia barwy o ok. 0.4%, co jest niemożliwe do wykrycia przez ludzki wzrok bez specjalistycznych narzędzi statystycznych.
- **Efektywność:**
  Algorytm pracuje w czasie liniowym O(n) względem liczby pikseli, co zapewnia szybkie działanie nawet dla dużych zdjęć.
- **Uniwersalność nośnika:**
  Jako "kontener" może służyć dowolne zdjęcie od wakacyjnej fotografii po zwykły zrzut ekranu.

### Wady i ograniczenia:

- **Brak odporności na przekształcenia:**
  Jakakolwiek edycja obrazu, czyli np. zmiana rozmiaru, przycięcie, zmiana jasności, a przede wszystkim zapis do stratnego formatu JPG niszczy ukrytą informację.
- **Podatność na analizę statystyczną:**

Choć zmiany są niewidoczne gołym okiem profesjonalna analiza histogramu obrazu mogłaby wykazać anomalie w rozkładzie najmniej znaczących bitów (tzw. steganaliza).

- **Bezpieczeństwo surowych danych:**
  Program ukrywa tekst jawny. W przypadku wykrycia faktu ukrycia danych osoba trzecia może je natychmiast odczytać.

## Sugestie dalszego rozwoju:

W celu podniesienia poziomu bezpieczeństwa aplikację można rozbudować o:

1. Szyfrowanie tekstu (np. AES) przed jego ukryciem w obrazie.
2. Rozproszenie bitów. Zamiast zapisywać bity jeden po drugim można użyć generatora liczb pseudolosowych z ziarnem (kluczem), aby rozsiać bity wiadomości po całej powierzchni obrazu co utrudnia steganalizę.
3. Wykorzystanie wszystkich kanałów. Użycie bitów LSB również w kanałach G (Green) i B (Blue) potroiłoby pojemność nośnika.